

QC/HPC Resource Allocation/Scheduling

Squillante and Tantawi and potentially others

January 2026

1 Introduction

TBA

2 Technical Preliminaries

TBA

2.1 Light-Heavy Traffic Interpolation

The light-heavy traffic interpolation approximation can be an effective method for analyzing otherwise intractable queueing systems. The central idea is to approximate some quantity of interest in the system $f(\rho)$ with an estimate $\tilde{f}(\rho)$ of the form

$$\tilde{f}(\rho) = \frac{a_n \rho^n + a_{n-1} \rho^{n-1} + \dots + a_1 \rho + a_0}{d(\rho)},$$

where $d(\rho)$ is an appropriately chosen normalizing denominator and $\{a_i\}_{i=0}^n$ are coefficients to be determined from $n + 1$ matching conditions. These involve the $n + 1$ derivatives of $f(\rho)$ at 0, i.e.,

$$\left. \frac{d^i \tilde{f}}{d\rho^i} \right|_{\rho=0} = \left. \frac{d^i f}{d\rho^i} \right|_{\rho=0}, \quad 0 \leq i \leq n,$$

or, equivalently, the n derivatives of $f(\rho)$ at 0 and a heavy traffic limit of the form

$$\lim_{\rho \rightarrow 1} d(\rho) f(\rho) = h,$$

where h is a positive and finite constant. The availability of the light traffic derivatives $d^i f / d\rho^i (\rho = 0)$ for all i fully determines $f(\rho)$ in terms of its Taylor expansion, and therefore provides a way to obtain an exact expression $\tilde{f}(\rho) = f(\rho)$. In most practical cases, however, only a few derivatives are available, say n , together with a heavy traffic limit. One

must thus rely on this partial information which yields an n^{th} degree polynomial in ρ as an approximation.

The calculation of the derivatives at zero for the quantity of interest is clearly a focal point of the light-heavy traffic approximation. This is made possible within the context of Poisson (or Poisson-driven) arrival processes by the light traffic theory rigorously developed in [27], where the authors furnish expressions that, in principle, enable us to calculate the derivative of any order n . Unfortunately, in our particular situation, the calculations quickly become extremely involved and we therefore have to restrict ourselves to the 0^{th} and 1^{st} order derivatives. Nevertheless, an a posteriori comparison of the resulting approximations with simulations strongly suggests that this quadratic interpolation is more than sufficient. Thus, following [27], we denote by $\widehat{f}(\emptyset)$ the expected value of f for a customer that joins an empty system and we denote by $\widehat{f}(\{t\})$ the conditional expected value of f for a customer that joins the system at time 0, given that exactly one other arrival has occurred at time t . Reiman and Simon show that

$$f(\rho = 0) = \widehat{f}(\emptyset) \quad (1)$$

and, in the case of Poisson arrivals,

$$\left. \frac{df}{d\rho} \right|_{\rho=0} = \mu \int_{-\infty}^{+\infty} [\widehat{f}(\{t\}) - \widehat{f}(\emptyset)] dt, \quad (2)$$

where $1/\mu$ denotes (as usual) the expected value of the customer service times.

Whenever the common choice $d(\rho) = \mu(1 - \rho)$ is appropriate, it is eventually possible to exploit (1), (2) and knowledge of a heavy traffic limit $\lim_{\rho \rightarrow 1} (1 - \rho)f(\rho) = h$, $0 < h < \infty$, to obtain a quadratic approximation $\widetilde{f}(\rho)$ for the quantity of interest.

We note that this light-heavy traffic interpolation has been shown to yield the exact solution obtained by Nelson and Tantawi [22] for the case where $\mu_Q^P = \mu_G^P$.

3 Mathematical Models

TBA

4 Mathematical Analysis of Computational Phases in Isolation

We first consider in isolation the two main computational phases of quantum-centric supercomputing applications, namely the sequential computational phase and the parallel computational phase.

4.1 Sequential Computational Phase

When considered in isolation, the sequential computational phase of quantum-centric supercomputing applications comprises with probability p_o a quantum computational component

executed on a system QPU followed by a classical computational component executed on a system GPU and comprises with probability $1 - p_o$ a classical computational component executed on a system GPU followed by a quantum computational component executed on a system QPU. The corresponding stochastic network consists of an open Jackson queueing network [4, 13] in which the arrivals to each queue i follow a Poisson process with rate γ_i and the service times for each queue i follow an exponential distribution with rate $\tilde{\mu}_i$. Depending upon the type of processing unit represented by queue i , the service rate $\tilde{\mu}_i$ can model a QPU with speed S_Q and mean service time μ_Q^{-1} where $\tilde{\mu}_i = S_Q \mu_Q$ or model a GPU with speed S_G and mean service time μ_G^{-1} where $\tilde{\mu}_i = S_G \mu_G$.

Such stochastic networks have received considerable attention in the research literature; see, e.g., [4, 13, 14, 16]. It is generally well known that these stochastic networks have a product-form solution, which implies that each queue of the stochastic network can be analyzed separate from the rest of the network. We therefore have that the expected sojourn time T_i of the sequential computational phases of jobs in isolation at queue i of the stochastic network is given by

$$\mathbb{E}[T_i] = \frac{1}{\tilde{\mu}_i - \gamma_i}. \quad (3)$$

The expected overall sojourn time T of the sequential computational phases of jobs in isolation is then obtained as the appropriate sum of the expected sojourn times at each of the queues. Although we primarily focus on $N = 2$ alternating sequential quantum and classical computational components, the corresponding results for a general number N of sequential computational phases would be obtained in a similar manner.

5 Parallel Computational Phase

When considered in isolation, the parallel computational phase of quantum-centric supercomputing applications comprises a single parallel pair of quantum and classical computational components with the former executed on a system QPU and the latter executed on a system GPU. The corresponding stochastic network consists of a fork-join queueing network with two parallel servers [10, 11] in which the arrivals follow a Poisson process with rate γ and the service times for each of the two queues i follow an exponential distribution with rate $\tilde{\mu}_i$. Here the service rate $\tilde{\mu}_1$ of the first of the two queues models a QPU with speed S_Q and mean service time μ_Q^{-1} where $\tilde{\mu}_1 = S_Q \mu_Q$ and the service rate $\tilde{\mu}_2$ of the second of the two queues models a GPU with speed S_G and mean service time μ_G^{-1} where $\tilde{\mu}_2 = S_G \mu_G$.

Hence, in both cases we have the traffic intensities $\rho_Q^P = \lambda / (S_Q \mu_Q^P)$ and $\rho_G^P = \lambda / (S_G \mu_G^P)$. This fork-join queueing network has received considerable attention in the research literature; see, e.g., [3, 10, 11, 20, 22, 26, 28].

6 Introduction

We consider quantum-centric supercomputing environments that comprise a combination of both quantum computing (QC) and high-performance computing (HPC).

{Asser: Background material: Expand}

- Integrating quantum computing and HPC. [6, 19, 30, 31]
- Estimating quantum execution time. [18]
- Several quantum algorithms are promising to solve problems more efficiently than possible on classical computers. However, various complex pre- and post-processing tasks have to be performed when executing a quantum circuit, which requires immense mathematical and technical knowledge. [32]
- Patterns of execution. [12]
- Hybrid classical-quantum optimization. [1]

7 Quantum-Centric Supercomputing Application Model

The basic model of a QC/HPC application consists of a simple directed acyclic graph, as depicted on the left-hand side of Figure 1. We define the *dag-level* of the directed acyclic graph as being the set of nodes that have the same constraints before they can be executed; e.g., dag-level 1 comprises node “a”, dag-level 2 comprises nodes “b” and “c”, dag-level 3 comprises node “d”, and dag-level 4 comprises node “e”. Hence, the computational phase of an QC/HPC application corresponding to each dag-level in the graph consists of either a single parallel pair of quantum and classical computational components, as depicted by nodes “b” and “c” on the left-hand side of Figure 1, or a single sequence of quantum and classical computational components, as depicted by node “d” on the left-hand side of Figure 1. The former parallel pair of quantum and classical computational components is illustrated by the quantum steps and classical steps in (a) of the right-hand side of Figure 1; and the latter sequence of quantum and classical computational components is illustrated by the quantum steps and classical steps in (b) of the right-hand side of Figure 1, where such a sequence of computational components can start with either a quantum or classical component and end with either a quantum or classical component. Although not a strict requirement, we envision that an application starts with a classical component to initiate the computations and a classical component to terminate the computations, where nodes “a” and “e” on the left-hand side of Figure 1 represent a single classical computational component.

Each quantum computational component on the right-hand side of Figure 1 can consist of parallel quantum computations requiring execution on $M_Q \geq 1$ quantum processing units (QPUs); and similarly, each classical computational component on the right-hand side of Figure 1 can consist of parallel classical computations requiring execution on $M_G \geq 1$ graphical

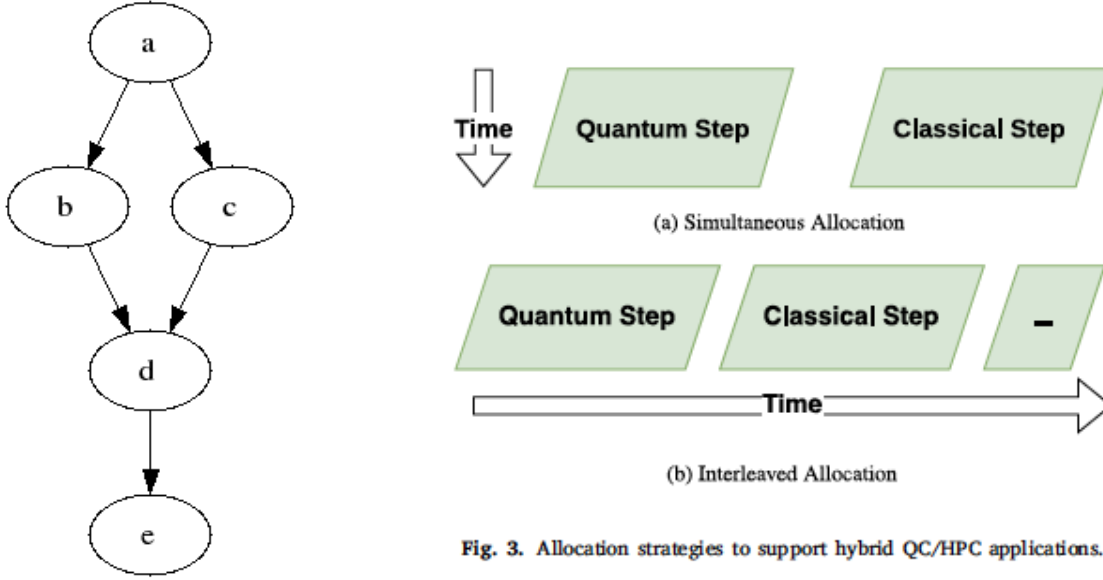


Fig. 3. Allocation strategies to support hybrid QC/HPC applications.

Figure 1: Abstraction of QC/HPC Applications. Left: Directed acyclic graph; Right: Fig. 3 from [30] on “Allocation strategies to support hybrid QC/HPC applications.”

processing units (GPUs). Analogously, the sequence of quantum and classical computational components in (b) on the right-hand side of Figure 1 can consist of $N \geq 1$ alternating quantum and classical computational components beginning with either a quantum or classical component. We note that M_Q , M_G and N can take on different values in different nodes of the directed acyclic graph. When $N = 1$, the corresponding node in the directed acyclic graph consists of either a single quantum or a single classical computational component.

As our initial starting point, we consider two classes of QC/HPC applications with different structures for the computational phases represented in the directed acyclic graph. The first class, called *parallel-sequential*, consists primarily of a parallel computational phase followed by a sequential computational phase as depicted in the directed acyclic graph on the left-hand side of Figure 1; a simplified version of this application class consists solely of the parallel computational phase in which node “d” is omitted. The second class, called *sequential-parallel*, consists primarily of a sequential computational phase followed by a parallel computational phase as depicted in the directed acyclic graph on the left-hand side of Figure 1 where the position of nodes “b” and “c” in the graph are swapped with that of node “d” (i.e., node “a” points to node “d” which points to both nodes “b” and “c” which each point to node “e”); a simplified version of this application class consists solely of the sequential computational phase in which nodes “b” and “c” are omitted.

8 Quantum-Centric Supercomputing System Model

An application arrives to the QC/HPC supercomputing center according to a Poisson process with rate λ . With probability $p_{\parallel}$, the application belongs to the first class of parallel-sequential applications; and with probability $1 - p_{\parallel}$, the application belongs to the second class of sequential-parallel applications. For simplicity, as our starting point, we will consider $M_Q = M_G = 1$ for all parallel computational phases and we will consider $N = 2$ for all sequential computational phases. With probability $p_{\circ}$, a sequential phase starts with a quantum computational component and with probability $1 - p_{\circ}$ the sequential phase starts with a classical computational component. In both the parallel-sequential and sequential-parallel classes, the service times for a quantum computational component of the parallel phase and sequential phase on QPUs respectively follow an exponential distribution with rate μ_Q^P and μ_Q^S . Similarly, the service times for a classical computational component of the parallel phase and sequential phase on GPUs respectively follow an exponential distribution with rate μ_G^P and μ_G^S . Nodes “a” and “e” are classical computational components executed on CPUs whose service times follow an exponential distribution with rates μ_B and μ_E , respectively, such that $\mu_B, \mu_E \gg \mu_Q^P, \mu_Q^S, \mu_G^P, \mu_G^S$.

To start, we consider three different instance of our workload model. The first workload instance, called *balanced*, consists of the case where $\mu_Q^P = \mu_G^P$ and $\mu_Q^S = \mu_G^S$; the second workload instance, called *quantum-dominant*, consists of the case where $\mu_Q^P < \mu_G^P$ and $\mu_Q^S < \mu_G^S$; and the third workload instance, called *classical-dominant*, consists of the case where $\mu_Q^P > \mu_G^P$ and $\mu_Q^S > \mu_G^S$. For simplicity, and again as a starting point, we can consider a single QPU with speed S_Q and a single GPU with speed S_G ; thus the service times for the quantum and classical computational components follow an exponential distribution with rates $S_Q\mu_Q^P, S_Q\mu_Q^S$ and $S_G\mu_G^P, S_G\mu_G^S$, respectively. We note that the queueing performance of a system with S_Q QPUs operating at unit speed converges to the queueing performance of our assumed single QPU with speed S_Q asymptotically as the quantum system utilization $\rho_Q \rightarrow 1$; and similarly for a system with S_G GPUs operating at unit speed asymptotically converging to our assumed single GPU with speed S_G as the classical system utilization $\rho_G \rightarrow 1$.

9 Performance Analysis

The queueing performance measures of interest concern the system utilization ρ_Q of the single QPU, the system utilization ρ_G of the single GPU, the sojourn time T_{PS} of the parallel-sequential applications, and the sojourn time T_{SP} of the sequential-parallel applications.

Let us first consider some simplifying instances of the quantum-centric supercomputing system model in Section 8. For each of these simplified settings, we assume that the computational requirements of nodes “a” and “e” on the left-hand side of Figure 1 are null.

9.1 Simplified Model: Fork-Joint Queueing Network of Quantum and Classical Computational Components

In the balanced workload case with homogeneous service rates $\mu = S_Q\mu_Q^P = S_G\mu_G^P$, it is well known that the expected queue length and expected sojourn time are given by [22]

$$\mathbb{E}[N_{FJ}] = \frac{\lambda(12 - \rho)}{8(\mu - \lambda)} \quad \text{and} \quad \mathbb{E}[T_{FJ}] = \frac{12 - \rho}{8(\mu - \lambda)}, \quad (4)$$

where $\rho = \lambda/\mu$. On the other hand, the two cases of unbalanced workloads with heterogeneous service rates, namely quantum-dominant workloads where $S_Q\mu_Q^P < S_G\mu_G^P$ and classical-dominant workloads where $S_G\mu_G^P < S_Q\mu_Q^P$, represent difficult queueing-theoretic problems. Let $\mu = \min\{S_Q\mu_Q^P, S_G\mu_G^P\}$ and $\bar{\mu} = \max\{S_Q\mu_Q^P, S_G\mu_G^P\}$. We note that, asymptotically as $(1/\bar{\mu}) \rightarrow 0$, the fork-join queue reduces to the corresponding M/M/1 queue with service rate μ whose expected queue length and expected sojourn time are given by [2, 15]

$$\mathbb{E}[N_{M/M/1}] = \frac{\rho}{1 - \rho} \quad \text{and} \quad \mathbb{E}[T_{M/M/1}] = \frac{1/\mu}{1 - \rho}, \quad (5)$$

where $\rho = \lambda/\mu$. More generally, however, there are no known exact expressions of the expected queue length and expected sojourn time for the fork-join queueing network with two servers and an unbalanced workload.

To investigate this queueing-theoretic problem, we first derive an approach based on a combination of phase-type distributions and known results for the M/G/1 queue in the special case where both GPU and QPU are reserved until both quantum and classical computational components are fully executed. Consider a continuous-time Markov process on the state space $\mathcal{S} := \{(q, g) \mid q \in \{0, 1\}, g \in \{0, 1\}\}$, and let $y_k \in \mathcal{S}$ be a reverse lexicographic ordering of the elements of $\mathcal{S}$, $k \in K$. Here, the state $(1, 1)$ represents the start of service for a fork-join job in which both the QPU and GPU are busy, state $(1, 0)$ represents the fork-join service in which the QPU is busy and the GPU is idle, state $(0, 1)$ represents the fork-join service in which the QPU is idle and the GPU is busy, and state $(0, 0)$ represents the completion of the fork-join service in which both the QPU and GPU are idle. Moreover, the ordering y_k is given by $(1, 1)$, $(1, 0)$, $(0, 1)$, $(0, 0)$, and thus $K = 4$. Following this lexicographic ordering, the infinitesimal generator matrix $\mathbf{Q}_{FJ}$ of the Markov process is given by

$$\mathbf{Q}_{FJ} = \begin{bmatrix} \tilde{\mathbf{Q}} & \tilde{\mathbf{q}} \\ \mathbf{0} & \mathbf{0} \end{bmatrix}, \quad \tilde{\mathbf{Q}} = \begin{bmatrix} -(\mu_G^P + \mu_Q^P) & \mu_G^P & \mu_Q^P \\ 0 & -\mu_Q^P & 0 \\ 0 & 0 & -\mu_G^P \end{bmatrix}, \quad \tilde{\mathbf{q}} = \begin{bmatrix} 0 \\ \mu_Q^P \\ \mu_G^P \end{bmatrix}, \quad (6)$$

together with the initial probability vector $(\boldsymbol{\alpha}, \alpha_{00})$ where $\boldsymbol{\alpha} = [1, 0, 0]$ and $\alpha_{00} = 0$. The distribution of the time to absorption in state $(0, 0)$ upon starting in state $(1, 1)$ with probability 1 defines the phase-type distribution with representation $(\boldsymbol{\alpha}, \mathbf{Q}_{FJ})$ for the service time of a job in the fork-join queueing network.

Let F be a random variable that follows the foregoing phase-type distribution for the service times of the special case of a fork-join queueing network. Then the distribution function and density function of F is given by

$$F(y) = 1 - \alpha \exp(\tilde{\mathbf{Q}}y)\mathbf{e}, \quad y \in \mathbb{R}_{\geq 0}, \quad \text{and} \quad f(y) = \alpha \exp(\tilde{\mathbf{Q}}y)\tilde{\mathbf{q}}, \quad y \in \mathbb{R}_{> 0},$$

where $\exp(\mathbf{Y}) := \sum_{j=0}^{\infty} \frac{1}{j!} \mathbf{Y}^j$. It is well known that the first two moments of the phase-type random variable F can be expressed as

$$\mathbb{E}[F] = -\alpha \tilde{\mathbf{Q}}^{-1} \mathbf{e} \quad \text{and} \quad \mathbb{E}[F^2] = 2\alpha \tilde{\mathbf{Q}}^{-2} \mathbf{e}. \quad (7)$$

To simplify notation, we shall simply use μ_Q and μ_G to respectively denote μ_Q^P and μ_G^P throughout the rest of this subsection. From (6) and direct calculation, we obtain

$$\tilde{\mathbf{Q}}^{-1} = \begin{bmatrix} -\frac{1}{\mu_G + \mu_Q} & -\frac{\mu_G}{\mu_Q(\mu_G + \mu_Q)} & -\frac{\mu_Q}{\mu_G(\mu_G + \mu_Q)} \\ 0 & -\frac{1}{\mu_Q} & 0 \\ 0 & 0 & -\frac{1}{\mu_G} \end{bmatrix},$$

$$\tilde{\mathbf{Q}}^{-2} = \begin{bmatrix} \frac{1}{(\mu_G + \mu_Q)^2} & \frac{\mu_G \mu_Q + \mu_G(\mu_G + \mu_Q)}{\mu_Q^2(\mu_G + \mu_Q)^2} & \frac{\mu_Q \mu_G + \mu_Q(\mu_G + \mu_Q)}{\mu_G^2(\mu_G + \mu_Q)^2} \\ 0 & \frac{1}{\mu_Q^2} & 0 \\ 0 & 0 & \frac{1}{\mu_G^2} \end{bmatrix},$$

which together with (7) yields

$$\mathbb{E}[F] = \frac{\mu_G^2 + \mu_G \mu_Q + \mu_Q^2}{\mu_G \mu_Q (\mu_G + \mu_Q)} \quad \text{and} \quad \mathbb{E}[F^2] = \frac{2(\mu_G^4 + \mu_G^2 \mu_Q^2 + \mu_Q^4) + 4(\mu_G^3 \mu_Q + \mu_G \mu_Q^3)}{\mu_G^4 \mu_Q^2 + 2\mu_G^3 \mu_Q^3 + \mu_G^2 \mu_Q^4}. \quad (8)$$

Next, we derive the desired expected queue length and expected sojourn time by combining the first two moments of the service time distribution for the special case of a fork-join queueing network with results for the M/G/1 queue. More precisely, it is well known that the expected queue length and expected sojourn time for the M/G/1 queue are given by [2, 15]

$$\mathbb{E}[N] = \lambda \mathbb{E}[F] + \frac{\lambda^2 \mathbb{E}[F^2]}{2(1 - \lambda \mathbb{E}[F])} \quad \text{and} \quad \mathbb{E}[T] = \mathbb{E}[F] + \frac{\lambda \mathbb{E}[F^2]}{2(1 - \lambda \mathbb{E}[F])}. \quad (9)$$

Upon substitution of (8) in (9), we obtain

$$\mathbb{E}[N] = \lambda \frac{\mu_G^2 + \mu_G \mu_Q + \mu_Q^2}{\mu_G \mu_Q (\mu_G + \mu_Q)} + \frac{\lambda [2\lambda(\mu_G^4 + \mu_G^2 \mu_Q^2 + \mu_Q^4) + 4\lambda(\mu_G^3 \mu_Q + \mu_G \mu_Q^3)](\mu_G^2 \mu_Q + \mu_G \mu_Q^2)}{[2\mu_G^2(\mu_Q - \lambda) + 2\mu_Q^2(\mu_G - \lambda) - 2\lambda \mu_G \mu_Q](\mu_G^4 \mu_Q^2 + 2\mu_G^3 \mu_Q^3 + \mu_G^2 \mu_Q^4)}, \quad (10)$$

$$\mathbb{E}[T] = \frac{\mu_G^2 + \mu_G \mu_Q + \mu_Q^2}{\mu_G \mu_Q (\mu_G + \mu_Q)} + \frac{[2\lambda(\mu_G^4 + \mu_G^2 \mu_Q^2 + \mu_Q^4) + 4\lambda(\mu_G^3 \mu_Q + \mu_G \mu_Q^3)](\mu_G^2 \mu_Q + \mu_G \mu_Q^2)}{[2\mu_G^2(\mu_Q - \lambda) + 2\mu_Q^2(\mu_G - \lambda) - 2\lambda \mu_G \mu_Q](\mu_G^4 \mu_Q^2 + 2\mu_G^3 \mu_Q^3 + \mu_G^2 \mu_Q^4)}. \quad (11)$$

Since no fork-join job starts its execution before the previous fork-join job departs the queueing network in this special case where the GPU and QPU are reserved until both quantum and classical computational components are fully executed, the expected queue length in (10) and the expected sojourn time in (11) respectively represent upper bounds on the expected queue length and the expected sojourn time of the standard fork-join queueing network.

10 General Case I

We consider the instance of our quantum-centric supercomputing application and system models where each arriving job consists of an application that starts with a single classical component to be executed on the system CPU (namely, node “a” in the left-hand side of Figure 1), followed by either a parallel computational phase (namely, nodes “b” and “c” in the left-hand side of Figure 1) or a sequential computational phase (namely, node “d” in the left-hand side of Figure 1), followed by a single classical component to be executed on the system CPU (namely, node “e” in the left-hand side of Figure 1). Such an arriving job consists of only a parallel computational phase with probability $p_{\parallel}$ and consists of only a sequential computational phase with probability $1 - p_{\parallel}$. The former parallel computational phase comprises a quantum computational component executed on the system QPU in parallel with a classical computational component executed on the system GPU, and the latter sequential computational phase comprises a quantum computational component executed on the system QPU followed by a classical computational component executed on the system GPU with probability p_o , or a classical computational component executed on the system GPU followed by a quantum computational component executed on the system QPU with probability $1 - p_o$. We focus here on the case in which each job reserves the system CPU, QPU and GPU throughout its execution without any overlap with other jobs that wait in the system queue.

More formally, consider a continuous-time Markov process $\{X(t); t \in \mathbb{R}_+\}$ on the state space $\hat{\Omega} = \bigcup_{i=0}^{\infty} \hat{\Omega}_i$ where $\hat{\Omega}_0 := \{0\}$ and $\hat{\Omega}_i := \{(i, j, k, q, g) \mid i \in \mathbb{Z}_+, j \in \{P, S, S_{gq}, S_{qq}\}, k \in \{B, C, E\}, q \in \{0, 1\}, g \in \{0, 1\}\}$, for the nonnegative reals $\mathbb{R}_+ := \mathbb{R}_{\geq 0}$ with $\mathbb{R}_{\geq k} := [k, \infty)$, and nonnegative integers $\mathbb{Z}_+ := \mathbb{Z}_{\geq 0}$ with $\mathbb{Z}_{\geq k} := \{k, k+1, k+2, \dots\}$. Here, the state-vector variable i denotes the total number of jobs in the quantum-centric supercomputing system, j denotes whether the job in service consists of a parallel computational phase (P) or a sequential computational phase (S) that comprises a classical computational component followed by a quantum computational component (S_{gq}) or a quantum computational component followed by a classical computational component (S_{qq}), k denotes whether the job in service is currently executing its beginning phase (B) or ending phase (E) on the system CPU or executing its quantum/classical computational phases (C), q denotes whether or not the job in service is currently executing a quantum computational component on the system QPU, and g denotes whether or not the job in service is currently executing a classical computational component on the system GPU. We shall refer to the set $\hat{\Omega}_i$ as level i .

Define $[M] := \{1, \dots, M\}$, $M < \infty$. Let $x_{i,m} \in \hat{\Omega}_i$ be a lexicographic ordering of the elements of $\hat{\Omega}_i$, $i \in \mathbb{Z}_{\geq 1}$, $m \in [M]$; similarly, let $\hat{\Omega}_0 = \{x_0\}$. The ordering $x_{i,m}$ for level

$i \in \mathbb{Z}_{\geq 1}$ is then given by $(i, P, B, 0, 0)$, $(i, P, C, 0, 1)$, $(i, P, C, 1, 0)$, $(i, P, C, 1, 1)$, $(i, P, E, 0, 0)$, $(i, S, B, 0, 0)$, $(i, S_{gq}, C, 0, 1)$, $(i, S_{gq}, C, 1, 0)$, $(i, S_{gq}, C, 0, 1)$, $(i, S_{gq}, C, 1, 0)$, $(i, S, E, 0, 0)$, and thus $M = 11$. Following this lexicographic ordering, the infinitesimal generator matrix $\mathbf{Q}$ of the Markov process $\{X(t); t \in \mathbb{R}_+\}$ has the block tridiagonal structure

$$\mathbf{Q} = \begin{bmatrix} \mathbf{B}_{00} & \mathbf{B}_{01} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots \\ \mathbf{B}_{10} & \mathbf{B}_{11} & \mathbf{A}_1 & \mathbf{0} & \mathbf{0} & \cdots \\ \mathbf{0} & \mathbf{A}_{-1} & \mathbf{A}_0 & \mathbf{A}_1 & \mathbf{0} & \cdots \\ \mathbf{0} & \mathbf{0} & \mathbf{A}_{-1} & \mathbf{A}_0 & \mathbf{A}_1 & \ddots \\ \vdots & \vdots & \ddots & \ddots & \ddots & \ddots \end{bmatrix}, \quad (12)$$

where $\mathbf{B}_{00}$, $\mathbf{B}_{01}$, $\mathbf{B}_{10}$, $\mathbf{B}_{11}$ and $\mathbf{A}_i$, $i \in \{-1, 0, 1\}$, are finite matrices of dimensions 1×1 , $1 \times M$, $M \times 1$, $M \times M$ and $M \times M$, respectively. By definition, since $\mathbf{Q}$ is a generator matrix, we have that $\mathbf{B}_{01}$, $\mathbf{B}_{10}$, $\mathbf{A}_{-1}$ and $\mathbf{A}_1$ are nonnegative matrices while $\mathbf{B}_{00}$, $\mathbf{B}_{11}$ and $\mathbf{A}_0$ have nonnegative offdiagonal elements and strictly negative diagonal elements such that the row sums of $\mathbf{Q}$ equal 0. Moreover, we have that $\mathbf{B}_0 = \mathbf{B}_{00} + \mathbf{B}_{01}$, $\mathbf{B}_1 = \mathbf{B}_{10} + \mathbf{B}_{11} + \mathbf{A}_1$, and $\mathbf{A} = \mathbf{A}_{-1} + \mathbf{A}_0 + \mathbf{A}_1$ are generator matrices.

The matrices corresponding to the non-homogeneous boundary of the state space $\hat{\Omega}$ are given by $\mathbf{B}_{00} = [-\lambda]$,

$$\mathbf{B}_{01} = [\mathbf{B}_{01}^P \quad \mathbf{B}_{01}^S], \quad \mathbf{B}_{01}^P = [p_0 \lambda \quad 0 \quad 0 \quad 0 \quad 0], \quad \mathbf{B}_{01}^S = [(1-p_0)\lambda \quad 0 \quad 0 \quad 0 \quad 0], \quad (13)$$

$$\mathbf{B}_{10} = [\mathbf{B}_{10}^P \quad \mathbf{B}_{10}^S]^\top, \quad \mathbf{B}_{10}^P = [0 \quad 0 \quad 0 \quad 0 \quad \mu_E^P], \quad \mathbf{B}_{10}^S = [0 \quad 0 \quad 0 \quad 0 \quad 0 \quad \mu_E^S], \quad (14)$$

$$\mathbf{B}_{11} = \begin{bmatrix} \mathbf{B}_{11}^P & \mathbf{0} \\ \mathbf{0} & \mathbf{B}_{11}^S \end{bmatrix}, \quad \mathbf{B}_{11}^P = \begin{bmatrix} (\lambda, \mu_B^P) & 0 & 0 & \mu_B^P & 0 \\ 0 & (\lambda, \mu_G^P) & 0 & 0 & \mu_G^P \\ 0 & 0 & (\lambda, \mu_Q^P) & 0 & \mu_Q^P \\ 0 & \mu_Q^P & \mu_G^P & (\lambda, \mu_Q^P, \mu_G^P) & 0 \\ 0 & 0 & 0 & 0 & (\lambda, \mu_E^P) \end{bmatrix}, \quad (15)$$

$$\mathbf{B}_{11}^S = \begin{bmatrix} (\lambda, \mu_B^S) & (1-p_0)\mu_B^S & 0 & 0 & p_0\mu_B^S & 0 \\ 0 & (\lambda, \mu_G^S) & \mu_G^S & 0 & 0 & 0 \\ 0 & 0 & (\lambda, \mu_Q^S) & 0 & 0 & \mu_Q^S \\ 0 & 0 & 0 & (\lambda, \mu_G^S) & 0 & \mu_G^S \\ 0 & 0 & 0 & \mu_Q^S & (\lambda, \mu_Q^S) & 0 \\ 0 & 0 & 0 & 0 & 0 & (\lambda, \mu_E^S) \end{bmatrix} \quad (16)$$

where $(x, y) = -(x + y)$ and $(x, y, z) = -(x + y + z)$. Here, the matrix $\mathbf{B}_{10}$ defines the rate of transitions from states in $\hat{\Omega}_1$ to the state $\hat{\Omega}_0$, the matrix $\mathbf{B}_{01}$ defines the rate of transitions from the state $\hat{\Omega}_0$ to states in $\hat{\Omega}_1$, and the offdiagonal elements of the matrix $\mathbf{B}_{11}$ define the rate of transitions among states in $\hat{\Omega}_1$. Similarly, for the homogeneous portion of the state space $\hat{\Omega}$, the matrix $\mathbf{A}_{-1}$ defines the rate of transitions from states in $\hat{\Omega}_{i+1}$ to states in $\hat{\Omega}_i$, the matrix $\mathbf{A}_1$ defines the rate of transitions from states in $\hat{\Omega}_i$ to states in $\hat{\Omega}_{i+1}$, and the offdiagonal elements of the matrix $\mathbf{A}_0$ define the rate of transitions among states in $\hat{\Omega}_i$, $i \in \mathbb{Z}_{\geq 1}$. Hence, the matrices for the homogeneous portion of the state space are given by

$\mathbf{A}_1 = \lambda \mathbf{I}$, $\mathbf{A}_0 = \mathbf{B}_{11}$, and

$$\mathbf{A}_{-1} = \begin{bmatrix} \mathbf{A}_{-1}^{PP} & \mathbf{A}_{-1}^{PS} \\ \mathbf{A}_{-1}^{SP} & \mathbf{A}_{-1}^{SS} \end{bmatrix}, \quad (17)$$

$$\mathbf{A}_{-1}^{PP} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ p_0 \mu_E^P & 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{A}_{-1}^{PS} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ (1-p_0)\mu_E^P & 0 & 0 & 0 & 0 \end{bmatrix}, \quad (18)$$

$$\mathbf{A}_{-1}^{SP} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ p_0 \mu_E^S & 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{A}_{-1}^{SS} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ (1-p_0)\mu_E^S & 0 & 0 & 0 & 0 \end{bmatrix}, \quad (19)$$

where $\mathbf{I}$ denotes the identity matrix of appropriate dimension.

Following the lexicographic ordering of the elements of each level i of the state space, define

$$\boldsymbol{\pi} := (\boldsymbol{\pi}_0, \boldsymbol{\pi}_1, \boldsymbol{\pi}_2, \dots), \quad (20)$$

$$\boldsymbol{\pi}_0 := (\hat{\pi}(x_0)), \quad \hat{\pi}(x_0) := \lim_{t \rightarrow \infty} \mathbb{P}[X(t) = x_0], \quad (21)$$

$$\boldsymbol{\pi}_i := (\hat{\pi}(x_{i,1}), \dots, \hat{\pi}(x_{i,M})), \quad \hat{\pi}(x_{i,m}) := \lim_{t \rightarrow \infty} \mathbb{P}[X(t) = x_{i,m}], \quad i \in \mathbb{Z}_{\geq 1}, \quad m \in [M]. \quad (22)$$

Assuming the Markov process $\{X(t); t \in \mathbb{R}_+\}$ to be irreducible and ergodic, the limiting probability vector $\boldsymbol{\pi}$ is the stationary distribution of this stochastic process such that its invariant probability vector $\boldsymbol{\pi}$ exists and is uniquely determined as the solution of $\boldsymbol{\pi} = \boldsymbol{\pi} \mathbf{P}$ and $\boldsymbol{\pi} \mathbf{e} = 1$, where $\mathbf{e} = (1, \dots, 1)^\top$ [7, 17]. Let $\text{sp}(\mathbf{Y})$ denote the spectral radius of a matrix $\mathbf{Y}$, and let a solution $\mathbf{Y}$ be called a minimal nonnegative solution of a matrix equation if $\mathbf{0} \leq \mathbf{Y} \leq \mathbf{Z}$ for any other nonnegative solution $\mathbf{Z}$, with the inequalities holding componentwise. The stationary probability vector $\boldsymbol{\pi}$ of the process $\{X(t); t \in \mathbb{R}_+\}$ with generator $\mathbf{Q}$ then can be obtained from the following matrix-analytic results [Theorems 1.7.1 and 1.5.1] [23].

Theorem 1. *Suppose that the Markov process $\{X(t); t \in \mathbb{R}_+\}$ is irreducible and aperiodic with $\mathbf{Q}$ in the form of (38). Further suppose the generator matrix $\mathbf{A} = \mathbf{A}_{-1} + \mathbf{A}_0 + \mathbf{A}_1$ is irreducible. Then this stochastic process is positive recurrent if and only if $\text{sp}(\mathbf{R}) < 1$, where $\mathbf{R}$ is the minimal nonnegative solution of the quadratic matrix equation*

$$\mathbf{R}^2 \mathbf{A}_{-1} + \mathbf{R} \mathbf{A}_0 + \mathbf{A}_1 = \mathbf{0}. \quad (23)$$

Furthermore, there exists a positive vector $(\boldsymbol{\pi}_0, \boldsymbol{\pi}_1)$ such that

$$(\boldsymbol{\pi}_0, \boldsymbol{\pi}_1) \begin{bmatrix} \mathbf{B}_{00} & \mathbf{B}_{01} \\ \mathbf{B}_{10} & \mathbf{B}_{11} + \mathbf{R} \mathbf{A}_{-1} \end{bmatrix} = \mathbf{0}. \quad (24)$$

The remaining components of the invariant probability vector $\boldsymbol{\pi}$ are then given by

$$\boldsymbol{\pi}_{n+1} = \boldsymbol{\pi}_1 \mathbf{R}^n, \quad n \in \mathbb{Z}_+, \quad (25)$$

where $(\boldsymbol{\pi}_0, \boldsymbol{\pi}_1)$ is normalized by

$$\boldsymbol{\pi}_0 + \boldsymbol{\pi}_1(\mathbf{I} - \mathbf{R})^{-1} \mathbf{e} = 1. \quad (26)$$

We have above assumed the irreducibility and ergodicity of the Markov process $\{X(t); t \in \mathbb{R}_+\}$. It readily follows from the definition of $\{X(t); t \in \mathbb{R}_+\}$ and its generator matrix $\mathbf{Q}$ that the Markov process is irreducible and aperiodic. Then the stability conditions for the Markov process to be positive recurrent are provided in Theorem 1, under the additional assumption that the generator matrix $\mathbf{A} = \mathbf{A}_{-1} + \mathbf{A}_0 + \mathbf{A}_1$ is irreducible. The irreducibility of $\mathbf{A}$ can be readily verified, as the directed graph corresponding to $\mathbf{A}$ is strongly connected with transitions from state $(*, P, B, 0, 0)$ to state $(*, P, E, 0, 0)$ reaching every state $(*, P, *, *, *)$ in between, transitions from state $(*, S, B, 0, 0)$ to state $(*, S, E, 0, 0)$ reaching every state $(*, S_{gg}, *, *, *)$ and $(*, S_{qq}, *, *, *)$ in between, and transitions from states $(*, *, E, 0, 0)$ to states $(*, *, B, 0, 0)$ connecting the states across both application types. Since the generator matrix $\mathbf{A}$ of the Markov process $\{X(t); t \in \mathbb{R}_+\}$ is irreducible, we then have necessary and sufficient conditions for $\text{sp}(\mathbf{R}) < 1$ from the following matrix-analytic result (see Theorem (TO BE LOOKED UP) in [23]). The stochastic process is therefore positive recurrent if and only if (27) is satisfied.

Theorem 2. *Suppose that the Markov process $\{X(t); t \in \mathbb{R}_+\}$ is irreducible. Further suppose that the generator $\mathbf{A}$ is irreducible. Then $\text{sp}(\mathbf{R}) < 1$ if and only if*

$$\mathbf{y} \mathbf{A}_1 \mathbf{e} < \mathbf{y} \mathbf{A}_{-1} \mathbf{e}, \quad (27)$$

where $\mathbf{y}$ is the stationary probability vector of $\mathbf{A}$, i.e., the (unique) solution of $\mathbf{y} \mathbf{A} = \mathbf{0}$ and $\mathbf{y} \mathbf{e} = 1$.

10.0.1 Performance Measures

Once the stationary distribution is determined, various functions of the components of the invariant probability vector $\boldsymbol{\pi}$ can be used to obtain performance measures and quantities of interest. In particular, the tail distribution of the queue length process can be expressed as

$$\mathbb{P}[Q > z] = \sum_{n=z+1}^{\infty} \boldsymbol{\pi}_n \mathbf{e} = \boldsymbol{\pi}_1 \mathbf{R}^z (\mathbf{I} - \mathbf{R})^{-1} \mathbf{e}, \quad z \in \mathbb{Z}_+, \quad (28)$$

with the corresponding expectation given by

$$\mathbb{E}[Q] = \sum_{n=0}^{\infty} (n+1) \boldsymbol{\pi}_{n+1} \mathbf{e} = \boldsymbol{\pi}_1 (\mathbf{I} - \mathbf{R})^{-1} \mathbf{e} + \boldsymbol{\pi}_1 \mathbf{R} (\mathbf{I} - \mathbf{R})^{-2} \mathbf{e}. \quad (29)$$

The expected sojourn time of a job can then be calculated using Little's law [33] and (29), which yields

$$\mathbb{E}[T] = \lambda^{-1} (\boldsymbol{\pi}_1(\mathbf{I} - \mathbf{R})^{-1} \mathbf{e} + \boldsymbol{\pi}_1 \mathbf{R}(\mathbf{I} - \mathbf{R})^{-2} \mathbf{e}). \quad (30)$$

Each of the above performance measures specific to the parallel-sequential application class, namely Q_{PS} and T_{PS} , can be obtained by replacing the vector $\mathbf{e}$ in (28), (29) and (30) with the vector ν_{PS} whose ℓ^{th} component contains a 1 for the states (i, j, k, q, g) involving the processing of jobs from the parallel-sequential application class, i.e., states with $j = P$, and contains a 0 otherwise; in (30), we additionally have λ^{-1} replaced by $(\lambda p_{\parallel})^{-1}$. Similarly, each of the above performance measures specific to the sequential-parallel application class, namely Q_{SP} and T_{SP} , can be obtained by replacing the vector $\mathbf{e}$ in (28), (29) and (30) with the vector ν_{SP} whose ℓ^{th} component contains a 1 for the states (i, j, k, q, g) involving the processing of jobs from the sequential-parallel application class, i.e., states with $j = S$, and contains a 0 otherwise; in (30), we additionally have λ^{-1} replaced by $(\lambda(1 - p_{\parallel}))^{-1}$.

The asymptotic decay rate of the tail distribution of the queue length process is another measure of interest. Let η denote this decay rate, which is also called the caudal characteristic [24, 25]. Due to the irreducibility of the Markov process $\{X(t); t \in \mathbb{R}_+\}$ and the irreducibility of the generator matrix $\mathbf{A}$, we either have $\eta = \text{sp}(\mathbf{R})$ when the matrix $\mathbf{R}$ is irreducible, or otherwise have $\eta = \text{sp}(\mathbf{R}_{11})$ where, after appropriate row and column permutations,

$$\mathbf{R} = \begin{bmatrix} \mathbf{R}_{11} & \mathbf{R}_{12} \\ \mathbf{0} & \mathbf{R}_{22} \end{bmatrix}$$

with $\mathbf{R}_{11}$ irreducible and $\mathbf{R}_{22}$ strictly upper triangular. We then conclude that $\mathbf{R}^z = O(\eta^z)$, as $z \rightarrow \infty$. Let $\mathbf{u}$ and $\mathbf{v}$ be the left and right eigenvectors corresponding to η that are strictly positive and normalized by $\mathbf{u}\mathbf{e} = 1$ and $\mathbf{u}\mathbf{v} = 1$. From the properties of the Markov process $\{X(t); t \in \mathbb{R}_+\}$, we know that [29]

$$\mathbf{R}^z = \eta^z \mathbf{v} \cdot \mathbf{u} + o(\eta^z), \quad \text{as } z \rightarrow \infty,$$

which together with equation (25) yields

$$\boldsymbol{\pi}_{z+1} \mathbf{e} = \boldsymbol{\pi}_1 \mathbf{v} \eta^z + o(\eta^z), \quad \text{as } z \rightarrow \infty. \quad (31)$$

It then follows that

$$\mathbb{P}[Q \geq z + 1] = \frac{\boldsymbol{\pi}_1 \mathbf{v}}{1 - \eta} \eta^z + o(\eta^z), \quad \text{as } z \rightarrow \infty, \quad (32)$$

and thus

$$\lim_{z \rightarrow \infty} \frac{\mathbb{P}[Q \geq z + 1]}{\eta^z} = \frac{\boldsymbol{\pi}_1 \mathbf{v}}{1 - \eta}, \quad (33)$$

or equivalently

$$\mathbb{P}[Q \geq z + 1] \sim \frac{\boldsymbol{\pi}_1 \mathbf{v}}{1 - \eta} \eta^z, \quad \text{as } z \rightarrow \infty, \quad (34)$$

where $f(z) \sim g(z)$ denotes that $\lim_{z \rightarrow \infty} f(z)/g(z) = 1$. Hence, in addition to providing the stability criterion for the quantum-centric supercomputing system, η is indicative of the tail behavior of the stationary queue length distribution.

Note that the caudal characteristic can be obtained without having to first solve for the matrix $\mathbf{R}$ in (23). Define the matrix $\tilde{\mathbf{A}}(s) := \mathbf{A}_1 + s\mathbf{A}_0 + s^2\mathbf{A}_{-1}$, for $0 < s \leq 1$. When the generator matrix $\mathbf{A}$ is irreducible, this matrix $\tilde{\mathbf{A}}(s)$ is irreducible with nonnegative off-diagonal elements. Let $\chi(s)$ denote the spectral radius of the matrix $\tilde{\mathbf{A}}(s)$. Then, under the above assumptions, η is the unique solution in $(0,1)$ of the equation $\chi(s) = 0$. This solution can be directly computed, although more efficient methods have been developed [5].

Another set of performance metrics of interest concern the stationary utilization of the CPU, QPU and GPU composing the quantum-centric supercomputing system. For each level i , $i \in \mathbb{Z}_+$, the states with components $k \in \{B, E\}$ represent execution on the system CPU, the states with components $q = 1$ represent execution on the system QPU, and the states with components $g = 1$ represent execution on the system GPU. The corresponding stationary utilizations can then be expressed as

$$U_C = \sum_{n=0}^{\infty} \pi_{n+1} \nu_C = \pi_1 (\mathbf{I} - \mathbf{R})^{-1} \nu_C, \quad (35)$$

$$U_Q = \sum_{n=0}^{\infty} \pi_{n+1} \nu_Q = \pi_1 (\mathbf{I} - \mathbf{R})^{-1} \nu_Q, \quad (36)$$

$$U_G = \sum_{n=0}^{\infty} \pi_{n+1} \nu_G = \pi_1 (\mathbf{I} - \mathbf{R})^{-1} \nu_G, \quad (37)$$

respectively where the ℓ^{th} component of the vector ν_C contains a 1 if $k \in \{B, E\}$ in the corresponding state of the homogeneous portion and contains a 0 otherwise, the ℓ^{th} component of the vector ν_Q contains a 1 if $q = 1$ in the corresponding state of the homogeneous portion and contains a 0 otherwise, and the ℓ^{th} component of the vector ν_G contains a 1 if $g = 1$ in the corresponding state of the homogeneous portion and contains a 0 otherwise.

11 General Case II

Next, we consider the instance of our quantum-centric supercomputing application and system models where each arriving job consists of an application that starts with a single classical component to be executed on the system CPU (namely, node “a” Figure 1), followed by either a parallel-sequential application class or a sequential-parallel application class, followed by a single classical component to be executed on the system CPU (namely, node “e” in Figure 1). A parallel-sequential application class comprises a parallel computational phase (namely, nodes “b” and “c” in Figure 1) followed by a sequential computational phase (namely, node “d” in Figure 1), whereas a sequential-parallel application class comprises a sequential computational phase (namely, node “d” in Figure 1) followed by a parallel computational phase (namely, nodes “b” and “c” in Figure 1). Such an arriving job belongs to the

parallel-sequential application class with probability $p_{\parallel}$ and belongs to the sequential-parallel application class with probability $1 - p_{\parallel}$. Each parallel computational phase comprises a quantum computational component executed on the system QPU in parallel with a classical computational component executed on the system GPU, and each sequential computational phase comprises a quantum computational component executed on the system QPU followed by a classical computational component executed on the system GPU with probability p_o , or a classical computational component executed on the system GPU followed by a quantum computational component executed on the system QPU with probability $1 - p_o$. We continue to focus here on the case in which each job reserves the system CPU, QPU and GPU throughout its execution without any overlap with other jobs that wait in the system queue. The results of this section extend the matrix-analytic analysis of Section 10 to handle the case in which each job includes a combination of both a parallel and sequential computational phases in either order.

More formally, consider a continuous-time Markov process $\{X(t); t \in \mathbb{R}_+\}$ on the state space $\hat{\Omega} = \bigcup_{i=0}^{\infty} \hat{\Omega}_i$ where $\hat{\Omega}_0 := \{0\}$ and $\hat{\Omega}_i := \{(i, j_1, j_2, k, q, g) \mid i \in \mathbb{Z}_+, j_1 \in \{P, S\}, j_2 \in \{P_*, S_{gq}, S_{gg}, S_*\}, k \in \{B, C, E\}, q \in \{0, 1\}, g \in \{0, 1\}\}$, for the nonnegative reals $\mathbb{R}_+ := \mathbb{R}_{\geq 0}$ with $\mathbb{R}_{\geq k} := [k, \infty)$, and nonnegative integers $\mathbb{Z}_+ := \mathbb{Z}_{\geq 0}$ with $\mathbb{Z}_{\geq k} := \{k, k+1, k+2, \dots\}$. Here, the state-vector variable i denotes the total number of jobs in the quantum-centric supercomputing system, j_1 denotes whether the job in service belongs to the parallel-sequential application class (P) or the sequential-parallel application class (S), j_2 denotes whether the application in service is executing a parallel computational phase (P) or a sequential computational phase that comprises a classical computational component followed by a quantum computational component (S_{gq}) or a quantum computational component followed by a classical computational component (S_{gg}), k denotes whether the job in service is currently executing its beginning phase (B) or ending phase (E) on the system CPU or executing its quantum/classical computational phases (C), q denotes whether or not the job in service is currently executing a quantum computational component on the system QPU, and g denotes whether or not the job in service is currently executing a classical computational component on the system GPU. We shall refer to the set $\hat{\Omega}_i$ as level i .

Define $[M] := \{1, \dots, M\}$, $M < \infty$. Let $x_{i,m} \in \hat{\Omega}_i$ be a lexicographic ordering of the elements of $\hat{\Omega}_i$, $i \in \mathbb{Z}_{\geq 1}$, $m \in [M]$; similarly, let $\hat{\Omega}_0 = \{x_0\}$. The ordering $x_{i,m}$ for level $i \in \mathbb{Z}_{\geq 1}$ is then given by $(i, P, P, B, 0, 0)$, $(i, P, P, C, 0, 1)$, $(i, P, P, C, 1, 0)$, $(i, P, P, C, 1, 1)$, $(i, P, S_{gq}, C, 0, 1)$, $(i, P, S_{gq}, C, 1, 0)$, $(i, P, S_{gq}, C, 0, 1)$, $(i, P, S_{gq}, C, 1, 0)$, $(i, P, P, E, 0, 0)$, $(i, S, S, B, 0, 0)$, $(i, S, P, C, 0, 1)$, $(i, S, P, C, 1, 0)$, $(i, S, P, C, 1, 1)$, $(i, S, S_{gq}, C, 0, 1)$, $(i, S, S_{gq}, C, 1, 0)$, $(i, S, S_{gq}, C, 0, 1)$, $(i, S, S_{gq}, C, 1, 0)$, $(i, S, S, E, 0, 0)$, and thus $M = 18$. Following this lexicographic ordering, the infinitesimal generator matrix $\mathbf{Q}$ of the Markov process $\{X(t); t \in \mathbb{R}_+\}$ has the block tridiagonal structure

$$\mathbf{Q} = \begin{bmatrix} \mathbf{B}_{00} & \mathbf{B}_{01} & \mathbf{0} & \mathbf{0} & \mathbf{0} & \cdots \\ \mathbf{B}_{10} & \mathbf{B}_{11} & \mathbf{A}_1 & \mathbf{0} & \mathbf{0} & \cdots \\ \mathbf{0} & \mathbf{A}_{-1} & \mathbf{A}_0 & \mathbf{A}_1 & \mathbf{0} & \cdots \\ \mathbf{0} & \mathbf{0} & \mathbf{A}_{-1} & \mathbf{A}_0 & \mathbf{A}_1 & \ddots \\ \vdots & \vdots & \ddots & \ddots & \ddots & \ddots \end{bmatrix}, \quad (38)$$

where $\mathbf{B}_{00}$, $\mathbf{B}_{01}$, $\mathbf{B}_{10}$, $\mathbf{B}_{11}$ and $\mathbf{A}_i$, $i \in \{-1, 0, 1\}$, are finite matrices of dimensions 1×1 , $1 \times M$, $M \times 1$, $M \times M$ and $M \times M$, respectively. By definition, since $\mathbf{Q}$ is a generator matrix, we have that $\mathbf{B}_{01}$, $\mathbf{B}_{10}$, $\mathbf{A}_{-1}$ and $\mathbf{A}_1$ are nonnegative matrices while $\mathbf{B}_{00}$, $\mathbf{B}_{11}$ and $\mathbf{A}_0$ have nonnegative offdiagonal elements and strictly negative diagonal elements such that the row sums of $\mathbf{Q}$ equal 0. Moreover, we have that $\mathbf{B}_0 = \mathbf{B}_{00} + \mathbf{B}_{01}$, $\mathbf{B}_1 = \mathbf{B}_{10} + \mathbf{B}_{11} + \mathbf{A}_1$, and $\mathbf{A} = \mathbf{A}_{-1} + \mathbf{A}_0 + \mathbf{A}_1$ are generator matrices.

The matrices corresponding to the non-homogeneous boundary of the state space $\hat{\Omega}$ are given by $\mathbf{B}_{00} = [-\lambda]$,

$$\mathbf{B}_{01} = [\mathbf{B}_{01}^P \quad \mathbf{B}_{01}^S], \quad \mathbf{B}_{10} = [\mathbf{B}_{10}^P \quad \mathbf{B}_{10}^S]^\top, \quad (39)$$

$$\mathbf{B}_{01}^P = [p_{\parallel}\lambda \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0], \quad \mathbf{B}_{01}^S = [(1-p_{\parallel})\lambda \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0], \quad (40)$$

$$\mathbf{B}_{10}^P = [0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad \mu_E^P], \quad \mathbf{B}_{10}^S = [0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad 0 \quad \mu_E^S], \quad (41)$$

$$\mathbf{B}_{11} = \begin{bmatrix} \mathbf{B}_{11}^P & \mathbf{0} \\ \mathbf{0} & \mathbf{B}_{11}^S \end{bmatrix}, \quad \mathbf{B}_{11}^P = \begin{bmatrix} \mathbf{B}_{11}^{P_{pp}} & \mathbf{B}_{11}^{P_{ps}} \\ \mathbf{0} & \mathbf{B}_{11}^{P_{ss}} \end{bmatrix}, \quad \mathbf{B}_{11}^S = \begin{bmatrix} \mathbf{B}_{11}^{S_{ss}} & \mathbf{B}_{11}^{S_{sp}} \\ \mathbf{0} & \mathbf{B}_{11}^{S_{pp}} \end{bmatrix}, \quad (42)$$

$$\mathbf{B}_{11}^{P_{pp}} = \begin{bmatrix} (\lambda, \mu_B^P) & 0 & 0 & \mu_B^P \\ 0 & (\lambda, \mu_G^{PP}) & 0 & 0 \\ 0 & 0 & (\lambda, \mu_Q^{PP}) & 0 \\ 0 & \mu_Q^{PP} & \mu_G^{PP} & (\lambda, \mu_Q^{PP}, \mu_G^{PP}) \end{bmatrix}, \quad (43)$$

$$\mathbf{B}_{11}^{P_{ps}} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 \\ (1-p_{\circ})\mu_G^{PP} & 0 & 0 & p_{\circ}\mu_G^{PP} & 0 \\ (1-p_{\circ})\mu_Q^{PP} & 0 & 0 & p_{\circ}\mu_Q^{PP} & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad (44)$$

$$\mathbf{B}_{11}^{P_{ss}} = \begin{bmatrix} (\lambda, \mu_G^{PS}) & \mu_G^{PS} & 0 & 0 & 0 \\ 0 & (\lambda, \mu_Q^{PS}) & 0 & 0 & \mu_Q^{PS} \\ 0 & 0 & (\lambda, \mu_G^{PS}) & 0 & \mu_G^{PS} \\ 0 & 0 & \mu_Q^{PS} & (\lambda, \mu_Q^{PS}) & 0 \\ 0 & 0 & 0 & 0 & (\lambda, \mu_E^P) \end{bmatrix}, \quad (45)$$

$$\mathbf{B}_{11}^{S_{ss}} = \begin{bmatrix} (\lambda, \mu_B^S) & (1-p_{\circ})\mu_B^S & 0 & p_{\circ}\mu_B^S & 0 \\ 0 & (\lambda, \mu_G^{SS}) & \mu_G^{SS} & 0 & 0 \\ 0 & 0 & (\lambda, \mu_Q^{SS}) & 0 & 0 \\ 0 & 0 & 0 & (\lambda, \mu_G^{SS}) & 0 \\ 0 & 0 & 0 & \mu_Q^{SS} & (\lambda, \mu_Q^{SS}) \end{bmatrix}, \quad (46)$$

$$\mathbf{B}_{11}^{S_{sp}} = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & \mu_Q^{SS} & 0 \\ 0 & 0 & \mu_G^{SS} & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad \mathbf{B}_{11}^{S_{pp}} = \begin{bmatrix} (\lambda, \mu_G^{SP}) & 0 & 0 & \mu_G^{SP} \\ 0 & (\lambda, \mu_Q^{SP}) & 0 & \mu_Q^{SP} \\ \mu_Q^{SP} & \mu_G^{SP} & (\lambda, \mu_Q^{SP}, \mu_G^{SP}) & 0 \\ 0 & 0 & 0 & (\lambda, \mu_E^S) \end{bmatrix}, \quad (47)$$

where $(x, y) = -(x + y)$ and $(x, y, z) = -(x + y + z)$. Here, the matrix $\mathbf{B}_{10}$ defines the rate of transitions from states in $\hat{\Omega}_1$ to the state $\hat{\Omega}_0$, the matrix $\mathbf{B}_{01}$ defines the rate of transitions

from the state $\hat{\Omega}_0$ to states in $\hat{\Omega}_1$, and the offdiagonal elements of the matrix $\mathbf{B}_{11}$ define the rate of transitions among states in $\hat{\Omega}_1$. Similarly, for the homogeneous portion of the state space $\hat{\Omega}$, the matrix $\mathbf{A}_{-1}$ defines the rate of transitions from states in $\hat{\Omega}_{i+1}$ to states in $\hat{\Omega}_i$, the matrix $\mathbf{A}_1$ defines the rate of transitions from states in $\hat{\Omega}_i$ to states in $\hat{\Omega}_{i+1}$, and the offdiagonal elements of the matrix $\mathbf{A}_0$ define the rate of transitions among states in $\hat{\Omega}_i$, $i \in \mathbb{Z}_{\geq 1}$. Hence, the matrices for the homogeneous portion of the state space are given by $\mathbf{A}_1 = \lambda \mathbf{I}$, $\mathbf{A}_0 = \mathbf{B}_{11}$, and

$$\mathbf{A}_{-1} = \begin{bmatrix} \mathbf{A}_{-1}^{PP} & \mathbf{A}_{-1}^{PS} \\ \mathbf{A}_{-1}^{SP} & \mathbf{A}_{-1}^{SS} \end{bmatrix}, \quad \mathbf{A}_{-1}^{PP} = \mathbf{A}_{-1}^{SP} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ p_0 \mu_E^P & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad (48)$$

$$\mathbf{A}_{-1}^{PS} = \mathbf{A}_{-1}^{SS} = \begin{bmatrix} 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\ (1 - p_0) \mu_E^P & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{bmatrix}, \quad (49)$$

where $\mathbf{I}$ denotes the identity matrix of appropriate dimension.

Following the lexicographic ordering of the elements of each level i of the state space, define

$$\boldsymbol{\pi} := (\boldsymbol{\pi}_0, \boldsymbol{\pi}_1, \boldsymbol{\pi}_2, \dots), \quad (50)$$

$$\boldsymbol{\pi}_0 := (\hat{\pi}(x_0)), \quad \hat{\pi}(x_0) := \lim_{t \rightarrow \infty} \mathbb{P}[X(t) = x_0], \quad (51)$$

$$\boldsymbol{\pi}_i := (\hat{\pi}(x_{i,1}), \dots, \hat{\pi}(x_{i,M})), \quad \hat{\pi}(x_{i,m}) := \lim_{t \rightarrow \infty} \mathbb{P}[X(t) = x_{i,m}], \quad i \in \mathbb{Z}_{\geq 1}, \quad m \in [M]. \quad (52)$$

Assuming the Markov process $\{X(t); t \in \mathbb{R}_+\}$ to be irreducible and ergodic, the limiting probability vector $\boldsymbol{\pi}$ is the stationary distribution of this stochastic process such that its invariant probability vector $\boldsymbol{\pi}$ exists and is uniquely determined as the solution of $\boldsymbol{\pi} = \boldsymbol{\pi} \mathbf{P}$ and $\boldsymbol{\pi} \mathbf{e} = 1$, where $\mathbf{e} = (1, \dots, 1)^\top$ [7, 17]. Let $\text{sp}(\mathbf{Y})$ denote the spectral radius of a matrix $\mathbf{Y}$, and let a solution $\mathbf{Y}$ be called a minimal nonnegative solution of a matrix equation if $\mathbf{0} \leq \mathbf{Y} \leq \mathbf{Z}$ for any other nonnegative solution $\mathbf{Z}$, with the inequalities holding componentwise. The stationary probability vector $\boldsymbol{\pi}$ of the process $\{X(t); t \in \mathbb{R}_+\}$ with generator $\mathbf{Q}$ then can be obtained from the following matrix-analytic results [Theorems 1.7.1 and 1.5.1] [23].

12 General Case III

Now we consider the application workload model of Section 11 while relaxing the system model assumption that each job reserves the system CPU, QPU and GPU throughout its execution without any overlap with other jobs that wait in the system queue.

For arriving jobs that belong to the sequential-parallel application class comprising a sequential computational phase followed by a parallel computational phase, which occurs with probability $1 - p_{\parallel}$, we have a stochastic tandem queueing network where the sequential computational phase has a product-form solution — refer to Section ???. This is the case irrespective of whether the sequential computational phase comprises a quantum computational component executed on the system QPU followed by a classical computational component executed on the system GPU, which occurs with probability $p_{\circ}$, or a classical computational component executed on the system GPU followed by a quantum computational component executed on the system QPU, which occurs with probability $1 - p_{\circ}$. Moreover, from the BCMP theorem [4], we have that the stochastic departure process from the sequential computational phase is Poisson and thus the parallel computational phase can be solved as the corresponding fork-join queueing network — refer to Section 9.1. The solutions of each of these phases can then be combined to obtain the performance of jobs of this type arriving with probability $1 - p_{\parallel}$.

For arriving jobs that belong to the parallel-sequential application class comprising a parallel computational phase followed by a sequential computational phase, which occurs with probability $p_{\parallel}$, we have a fork-join queueing network that can be solved as discussed in Section 9.1. However, the departure process from this fork-join queueing network is no longer Poisson and therefore the sequential computational phase no longer has a product-form solution. We likely need to develop an approximation — one option could be to exploit the general approach in [9] which treats cases exactly like this.

References

- [1] A. Ajagekar, K. A. Hamoud, and F. You. Hybrid classical-quantum optimization techniques for solving mixed-integer programming problems in production scheduling. *IEEE Transactions on Quantum Engineering*, 3:1–16, 2022.
- [2] S. Asmussen. *Applied Probability and Queues*. Springer, second edition, 2003.
- [3] S. Balsamo, L. Donatiello, and N. Van Dijk. Bound performance models of heterogeneous parallel processing systems. *IEEE Transactions on Parallel and Distributed Systems*, 9(10):1041–1056, 1998.
- [4] F. Baskett, K. M. Chandy, R. R. Muntz, and F. G. Palacios. Open, closed, and mixed networks of queues with different classes of customers. *Journal of the Association for Computing Machinery*, 22(2):248–260, 1975.

- [5] N. G. Bean, J.-M. Li, and P. G. Taylor. Caudal characteristics of qbds with decomposable phase spaces. In G. Latouche and P. Taylor, editors, *Advances in Algorithmic Methods for Stochastic Models*, pages 37–55. Notable Publications, 2000.
- [6] T. Beck, A. Baroni, R. Bennink, G. Buchs, E. A. C. Pérez, M. Eisenbach, R. F. da Silva, M. G. Meena, K. Gottiparthi, P. Groszkowski, T. S. Humble, R. Landfield, K. Maheshwari, S. Oral, M. A. Sandoval, A. Shehata, I.-S. Suh, and C. Zimmer. Integrating quantum computing resources into scientific hpc ecosystems. *Future Generation Computer Systems*, 161:11–25, 2024.
- [7] P. Brémaud. *Markov Chains: Gibbs Fields, Monte Carlo Simulation and Queues*. Springer, second edition, 2020.
- [8] P. J. Burke. The output of a queueing system. *Operations Research*, 4(6):699–704, 1956.
- [9] A. B. Dieker and M. S. S. Soumyadip Ghosh. Optimal resource capacity management for stochastic networks. *Operations Research*, 65(1):221–241, 2017.
- [10] L. Flatto. Two parallel queues created by arrivals with two demands II. *SIAM Journal of Applied Mathematics*, 45:861–878, 1985.
- [11] L. Flatto and S. Hahn. Two parallel queues created by arrivals with two demands I. *SIAM Journal of Applied Mathematics*, 44:1041–1053, 1984.
- [12] D. Georg, J. Barzen, M. Beisel, F. Leymann, J. Obst, D. Vietz, B. Weder, and V. Yusupov. Execution patterns for quantum applications. In *ICSOF*T, pages 258–268, 2023.
- [13] J. R. Jackson. Jobshop-like queueing systems. *Management Science*, 10(1):131–142, 1963.
- [14] F. P. Kelly. *Reversibility and Stochastic Networks*. Wiley, 1979.
- [15] L. Kleinrock. *Queueing Systems Volume I: Theory*. John Wiley & Sons, 1975.
- [16] S. S. Lavenberg, editor. *Computer Performance Modeling Handbook*. Academic Press, 1983.
- [17] D. A. Levin, Y. Peres, and E. L. Wilmer. *Markov Chains and Mixing Times*. American Mathematical Society, second edition, 2017.
- [18] N. Ma and H. Li. Understanding and estimating the execution time of quantum circuits, 2025.
- [19] P. Mantha, F. J. Kiwit, N. Saurabh, S. Jha, and A. Luckow. Pilot-quantum: A quantum-hpc middleware for resource, workload and task management, 2025.

- [20] M. Mohanty, G. Gautam, V. Aggarwal, and P. Parag. Analysis of fork-join scheduling on heterogeneous parallel servers. *IEEE/ACM Transactions on Networking*, 32(6):4798–4809, 2024.
- [21] R. R. Muntz. Poisson departure process and queueing networks. Technical Report RC 4145, IBM Research, 1972.
- [22] R. D. Nelson and A. N. Tantawi. Approximate analysis of fork/join synchronization in parallel queues. *IEEE Transactions on Computers*, 37(6):739–743, June 1988.
- [23] M. F. Neuts. *Matrix-Geometric Solutions in Stochastic Models: An Algorithmic Approach*. 1981.
- [24] M. F. Neuts. The caudal characteristic curve of queues. *Advances in Applied Probability*, 18:221–254, 1986.
- [25] M. F. Neuts. *Structured Stochastic Matrices of M/G/1 Type and Their Applications*. Marcel Dekker, 1989.
- [26] V. Nguyen. The trouble with diversity: Fork-join networks with heterogeneous customer population. *Ann. Appl. Probab.*, 4(4):1–25, 1994.
- [27] M. I. Reiman and B. Simon. Open queueing systems in light traffic. *Mathematics of Operations Research*, 14(1):26–59, February 1989.
- [28] D. Schol, M. Vlasiou, and B. Zwart. Extreme values for the waiting time in large fork-join queues. *Queueing Systems*, 2025.
- [29] E. Seneta. *Non-Negative Matrices and Markov Chains*. Springer Verlag, second edition, 1981.
- [30] A. Shehata et al. Bridging paradigms: Designing for HPC-Quantum convergence. *Future Generation Computer Systems*, 174:107980, 2026.
- [31] M. Slys, P. Rydlichowski, K. Kurowski, O. Bacarreza, E. C. Gomez, Z. Chandani, B. Heim, P. Khalate, W. R. Clements, and J. Fletcher. Hybrid classical-quantum supercomputing: A demonstration of a multi-user, multi-qpu and multi-gpu environment, 2025.
- [32] B. Weder, U. Breitenbücher, F. Leymann, and K. Wild. Integrating quantum computing into workflow modeling and execution. In *2020 IEEE/ACM 13th International Conference on Utility and Cloud Computing (UCC)*, pages 279–291, 2020.
- [33] R. W. Wolff. *Stochastic Modeling and the Theory of Queues*. Prentice Hall, 1989.

A Optimal Capacity Planning

With the exception of the parallel computational phases comprising fork-join queues, the stochastic network of our quantum-centric supercomputing application and system models consists of an open Jackson queueing network [13]. Such stochastic networks are known to have a product-form solution, and thus the expected sojourn time T_i of a job at queue i of the stochastic network is given by

$$\mathbb{E}[T_i] = \frac{1}{\tilde{\mu}_i - \gamma_i}, \quad (53)$$

where $\tilde{\mu}_i = S_i \mu_i$ and γ_i is the arrival rate to queue i in this stochastic network excluding the fork-join queues; see, e.g., [14, 16].

Let us consider the case where arriving jobs belong solely to the sequential-parallel application class comprising a sequential computational phase followed by a parallel computational phase. We therefore have $M = 5$ queues with the fifth queue representing the fork-join queue. All but the last queue (i.e., the fork-join queue) of the resulting stochastic network has a product-form solution, and thus $\mathbb{E}[T_i]$ is given by (53) for $i \in [4]$ where $\gamma_1 = \gamma_2 = p_o \lambda$ and $\gamma_3 = \gamma_4 = (1 - p_o) \lambda$. From Burke's Theorem [8] and the quasireversibility of such stochastic networks [14, 21], we also have that the departure process from queue 2 and 4 are Poisson with rate $p_o \lambda$ and $(1 - p_o) \lambda$, respectively. The arrival process to the fork-join queue is therefore Poisson with rate λ , and thus the sojourn time of the fork-join queue can also be analyzed in isolation. For the balanced workload case with homogeneous service rates $\tilde{\mu}_5 = \mu_Q^{SP} = \mu_G^{SP}$, we then obtain

$$\mathbb{E}[T_5] = \frac{12 - \lambda/\tilde{\mu}_5}{8(\tilde{\mu}_5 - \lambda)}. \quad (54)$$

Let c_i denote the cost associated with each unit of resource capacity $\tilde{\mu}_i$ at queue $i \in [M]$ with $M = 5$. Define the cost and resource capacity vectors $\mathbf{c} := (c_1, \dots, c_M)$ and $\tilde{\boldsymbol{\mu}} := (\tilde{\mu}_1, \dots, \tilde{\mu}_M)$, respectively. The objective of our capacity planning optimization problem then is to minimize the expected stationary sojourn times weighted by a vector $\boldsymbol{\omega} := (\omega_1, \dots, \omega_M)$, subject to a budget constraint $\mathcal{C}$ and to the constraint that the stochastic network is stable. More precisely, our capacity planning optimization problem is formulated as follows.

$$\begin{aligned} \text{(OPT)} \quad & \min_{\tilde{\boldsymbol{\mu}} \in (0, \infty)^M} \sum_{i=1}^M \omega_i \mathbb{E}[T_i] \end{aligned} \quad (55)$$

$$\text{s.t.} \quad \langle \mathbf{c}, \tilde{\boldsymbol{\mu}} \rangle \leq \mathcal{C}, \quad \text{and} \quad \tilde{\mu}_i > \gamma_i, \quad \forall i \in [M]. \quad (56)$$

To determine the optimal capacity allocation vector $\tilde{\boldsymbol{\mu}}$ subject to the constraints (56), the optimization problem (OPT) can be solved via Lagrangian methods. Specifically, the Lagrangian associated with (OPT) is defined as

$$\sum_{i=1}^M \omega_i \mathbb{E}[T_i] + \alpha \left(\sum_{i=1}^M c_i \tilde{\mu}_i - \mathcal{C} \right), \quad (57)$$

where $\mathbb{E}[T_i]$, $i \in [M - 1]$, is given by (53), $\mathbb{E}[T_M]$ is given by (54), and α is the Lagrange multiplier. Taking the derivative of (57) with respect to $\tilde{\mu}_i$ for the first four queues and setting the result to zero, we have

$$\frac{\omega_i}{(\tilde{\mu}_i - \gamma_i)^2} = \alpha c_i \quad \Rightarrow \quad \tilde{\mu}_i = \gamma_i + \frac{1}{\sqrt{\alpha}} \sqrt{\frac{\omega_i}{c_i}}. \quad (58)$$

Similarly, taking the derivative of (57) with respect to $\tilde{\mu}_M$ for the fork-join queue and setting the result to zero, we obtain

$$\frac{\omega_M(12\tilde{\mu}_M^2 - 2\lambda\tilde{\mu}_M + \lambda^2)}{8\tilde{\mu}_M^2(\tilde{\mu}_M - \lambda)^2} = \alpha c_M, \quad (59)$$

which after detailed derivations leads to the following fourth-order polynomial, or quartic, equation of $\tilde{\mu}_M$:

$$\tilde{\mu}_M^4 - 2\lambda\tilde{\mu}_M^3 + \left(\lambda^2 - \frac{3\omega_M}{2\alpha c_M}\right)\tilde{\mu}_M^2 + \frac{\lambda\omega_M}{4\alpha c_M}\tilde{\mu}_M - \frac{\lambda^2\omega_M}{8\alpha c_M} = 0. \quad (60)$$

The solution of our capacity planning optimization problem can then be efficiently computed via known numerical methods together with (57) – (60).